14

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

HotSpot Kilit Şişmesi: Biased Locking, Lightweight Locking & Heavyweight Monitor

HotSpot Lock Optimization: Biased, Lightweight (Thin) & Heavyweight Lock Inflation

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                HOTSPOT KİLİT ŞİŞMESİ (LOCK INFLATION) AŞAMALARI │
└──────────────────────────────────────────────────────────────┘

1. BIASED LOCKING (Taraflı Kilit - Tarihsel / Tek Thread):
   [ Nesne Başlığı ──► Thread ID yazılır ]
   Kilit maliyeti = 0 (Sadece tek bir thread giriyorsa sıfır senkronizasyon maliyeti).

2. LIGHTWEIGHT LOCK (Hafif Kilit / Çekişmesiz Çoklu Thread):
   Farklı thread'ler giriyor ama AYNI ANDA DEĞİL (Alternating execution).
   [ Nesne Başlığı ──► Thread'in kendi Stack'indeki BasicObjectLock'a işaret eder (CAS) ]
   İşletim sistemi kernel'ına İNİLMEZ, tamamen user-space CAS ile çözülür!

3. HEAVYWEIGHT MONITOR (Ağır Kilit / Çekişmeli Durum):
   İki thread AYNI ANDA kilide saldırır! CAS başarısız olur.
   [ Nesne Başlığı ──► OS seviyesinde ObjectMonitor C++ nesnesine işaret eder ]
   Thread'ler OS kernel seviyesinde BLOCKED durumuna alınır ve uyutulur (Park).
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketteki toplantı odasını kiralama adımları gibidir: 1) Taraflı Kilit: Odayı hep aynı genel müdür kullanıyorsa, kapıya genel müdürün resmi asılır. Müdür kapıyı çalmadan doğrudan girer (Sıfır maliyet). 2) Hafif Kilit: Odayı ara sıra farklı çalışanlar kullanıyor ama hiç kimse birbiriyle çakışmıyor. Kapıya basit bir mandal konur (Kullanıcı seviyesinde CAS). 3) Ağır Kilit: İki yönetici aynı anda odaya girmeye çalışır ve kapıda kavga çıkar! Güvenlik (OS Kernel) çağrılır, yöneticilerden biri zorla bekleme salonuna oturtulup uyutulur (Context Switch maliyeti)!

🇬🇧 ENGLISH (ELI5)

HotSpot optimizes locking like an escalating conference room protocol: 1) Biased Locking: If Manager A is the sole user, their nameplate stays on the door; they walk in without checking locks (zero cost). 2) Lightweight (Thin) Locking: Different team members book the room, but their schedules never collide. They flip a simple physical door latch using atomic CAS in user space without security guards. 3) Heavyweight Monitor: Two directors arrive simultaneously and collide at the door threshold! Armed security (the OS kernel) intervenes, putting one director into forced sleep in the waiting lounge.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Kilit Optimizasyon Kademeleri: 1) Kilit Şişmesi (Inflation) Tek Yönlüdür: HotSpot'ta kilit Hafif'ten Ağır'a şiştiğinde (Inflatable), genellikle JVM yeniden başlatılana kadar ağır kalır. 2) Lightweight Locking (CAS Lock): Thread, kendi thread stack'inde bir `BasicObjectLock` açar ve nesnenin Mark Word'ünü CAS ile bu stack kaydına yönlendirir. Çekişme yoksa tek bir CAS ile kilit alınır ve bırakılır. 3) Spin-Locking (Uyarlamalı Dönme / Adaptive Spinning): Ağır kilide geçmeden önce HotSpot thread'i hemen uyutmaz; kısa bir süre döngüde (spin) bekler. Eğer kilidi tutan diğer thread hemen işini bitirirse, pahalı işletim sistemi context-switch maliyetinden kurtulunur! 4) Biased Locking Durumu: Modern JDK sürümlerinde (JDK 15+) Biased Locking, safepoint maliyetleri nedeniyle varsayılan olarak kapatılmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Lock Optimization Architecture: 1) One-Way Inflation: Once contention forces an object's Mark Word to point to an inflated C++ `ObjectMonitor`, deflation rarely occurs until safepoint GC phases. 2) Lightweight Locking via Stack Displaced Mark Word: A thread copies the target object's Mark Word to its local stack frame (Displaced Mark Word) and performs a CAS to store the stack address in the object header. If successful, acquisition completes with zero kernel transitions. 3) Adaptive Spinning: Before invoking heavy `pthread_mutex_lock` (which suspends the thread), the JVM spins the core for several clock cycles. If the holding thread releases during the spin, execution proceeds seamlessly without context-switch latency.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Kilit Çekişmesi ve Adaptif Spinning Simülasyonu
public class LockInflationDemo {
    private final Object monitor = new Object();
    private int counter = 0;

    // Çekişmesiz: Hafif Kilit (Lightweight CAS) olarak çalışır
    public void uncontendedOperation() {
        synchronized (monitor) {
            counter++;
        }
    }

    // Yüksek Çekişme: Ağır Kilit (Heavyweight Monitor Inflation) tetikler
    public void contendedOperation() {
        synchronized (monitor) {
            try {
                Thread.sleep(1); // Bilerek kilit içinde beklenir, diğerleri şişer!
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            counter++;
        }
    }

    void main() {
        var demo = new LockInflationDemo();
        demo.uncontendedOperation();
        IO.println("Hafif kilit başarıyla çalıştı: " + demo.counter);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Thread.sleep() kilit içindeyken kilidi bırakır mı?' Yanıt: 'KESİNLİKLE BIRAKMAZ! sleep() kilidi tutmaya devam ederken diğer tüm thread'leri kapıda bloke eder. Kilidi geçici olarak bırakan metot Object.wait()'tir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Crucial Interview Question: 'Does Thread.sleep() release held synchronized monitors?' Answer: 'ABSOLUTELY NOT! `Thread.sleep()` sleeps while retaining ownership of every single held lock, starving other threads. Only `Object.wait()` atomically releases the monitor.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HotSpot kilitleri çekişmeye göre Lightweight'ten Heavyweight'e şişer (Inflate).
- Adaptif dönme (Adaptive Spinning), thread uyutma (park) maliyetini önler.
- Thread.sleep() kilidi ASLA bırakmaz; Object.wait() ise kilidi bırakır.

🎯 Key Takeaways for the Interview

- HotSpot locks dynamically inflate from Lightweight CAS to Heavyweight OS monitors.
- Adaptive spinning attempts to acquire monitors during spin loops before sleeping.
- Thread.sleep() retains locks; Object.wait() atomically surrenders them.